PROBLEM 9 · SOLUTION

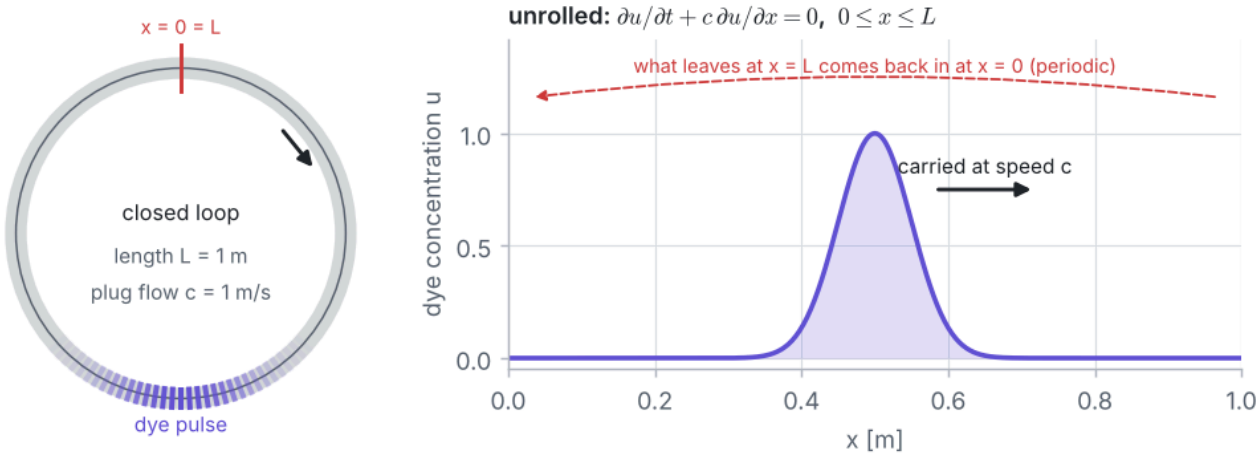
Advection of a pulse

Step-by-step solution with the Finite Difference Method

Plan. We follow the 7 pillars of CFD: geometry → grid → physics → numerical scheme → solution setup → compute → post-processing. All code below is plain Python (lists and loops only; `math` and `time` from Python itself). It is the code of the notebook `03_fdm_solver.ipynb`, where it can be run cell by cell.

Step 1 · Geometry and data

Water circulates round a closed loop of pipe, $L = 1$ m long, as a plug flow with speed $c = 1$ m/s, carrying a pulse of dye. Molecular diffusion of a dye in water ($D \approx 10^{-9}$ m²/s) spreads it by only about 0.05 mm in one second, so we neglect it: the dye is **only carried** by the flow. Unrolling the loop gives a straight domain $0 \le x \le L$ whose two ends are the same point: a **periodic** domain.



The loop and the unrolled, periodic domain.

```
L = 1.0          # length of the loop                       [m]
c = 1.0          # flow speed (plug flow)                   [m/s]
t_lap = L / c    # time for one lap                         [s]

x0 = 0.5         # centre of both pulses                    [m]
sigma = 0.05     # width (standard deviation) of the Gaussian [m]
width = 0.2      # width of the square pulse                [m]

print(f"one lap takes t = L/c = {t_lap:g} s")
```

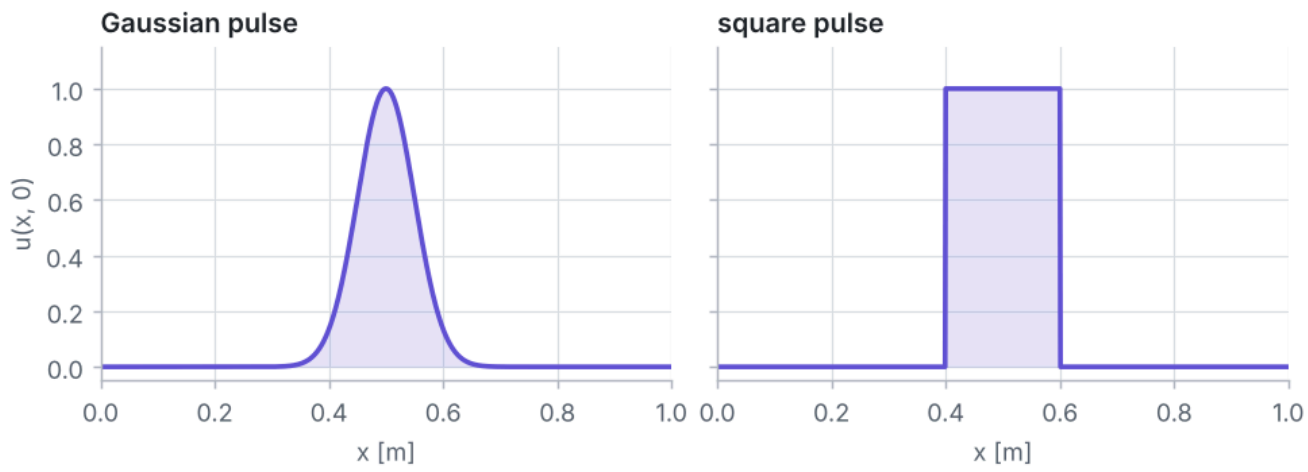
```
one lap takes t = L/c = 1 s
```

Two initial pulses, both centred at $x_0 = 0.5$ m and of height 1: a smooth **Gaussian** ($\sigma = 0.05$ m), to measure the order of accuracy, and a sharp **square pulse** (0.4 to 0.6 m), which contains waves of every length and shows what a scheme does to the shortest ones.

```
def gaussian(xx):
    """Smooth pulse, height 1, centred at x0."""
    return math.exp(-(xx - x0)**2 / (2 * sigma**2))


def square(xx):
    """Sharp pulse: 1 between x0 - width/2 and x0 + width/2, else 0."""
    return 1.0 if abs(xx - x0) <= width / 2 + 1e-9 else 0.0     # 1e-9: tolerance for round-off


pulses = {"Gaussian pulse": gaussian, "square pulse": square}
```



The two initial pulses.

Step 2 · The grid in space and time

Space. N intervals, $\Delta x = L/N$. Node N (at $x = L$) **is** node 0, so only the N nodes $i = 0, \ldots, N-1$ are stored. Every node has two neighbours, also at the ends: the west neighbour of node 0 is node $N-1$, the east neighbour of node $N-1$ is node 0. There are no boundary nodes and no boundary conditions.

Time. As in Problem 2, time steps Δt, $t^n = n\,\Delta t$, values u_i^n.

Courant number. In one time step the flow moves the dye by $c\,\Delta t$; in grid intervals this is

$$C = \frac{c\,\Delta t}{\Delta x}.$$

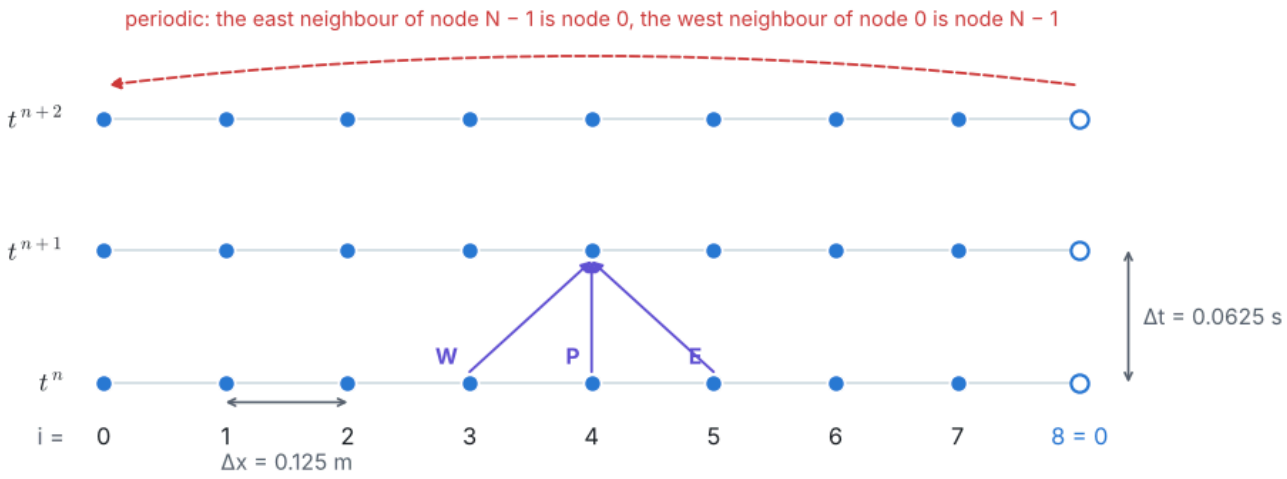
It plays the role of $r = \alpha\,\Delta t/\Delta x^2$ in Problem 2. We choose N and C; then $\Delta t = C\,\Delta x/c$, and one lap takes N/C steps.

```
N = 100                 # number of intervals = number of stored nodes
dx = L / N              # grid spacing [m]
x = []                  # positions of the nodes 0 .. N-1 (node N is node 0 again)
for i in range(N):
    x.append(i * dx)

C = 0.5                 # Courant number
dt = C * dx / c         # time step [s]
n_lap = round(t_lap / dt)

print(f"N = {N} intervals, dx = {dx:g} m, nodes x_0 = {x[0]:g} ... x_{N - 1} = {x[-1]:g} m")
print(f"C = {C:g}  ->  dt = C dx / c = {dt:g} s,  {n_lap} time steps for one lap")
```

```
N = 100 intervals, dx = 0.01 m, nodes x_0 = 0 ... x_99 = 0.99 m
C = 0.5  ->  dt = C dx / c = 0.005 s,  200 time steps for one lap
```



The periodic grid in x and t (drawn with $N = 8$, the grid of the whiteboard guide) and the three-point stencil.

Step 3 · Physics: the advection equation

A balance of the dye in a short piece of pipe (flux cu in, flux cu out, nothing else) gives the **linear advection equation**

$$\frac{\partial u}{\partial t} + c\,\frac{\partial u}{\partial x} = 0, \qquad u(0,t) = u(L,t), \qquad u(x,0) = u_0(x).$$

It is the convection–diffusion equation of Problem 5 with the time derivative put back and no diffusion, and it is the simplest **hyperbolic** equation. Along any line $x = x_0 + ct$,

$$\frac{d}{dt}\,u(x_0 + ct,\ t) = \frac{\partial u}{\partial t} + c\,\frac{\partial u}{\partial x} = 0,$$

so u is constant along these **characteristics**, and the **exact solution** is the initial profile shifted by ct (wrapped round the loop):

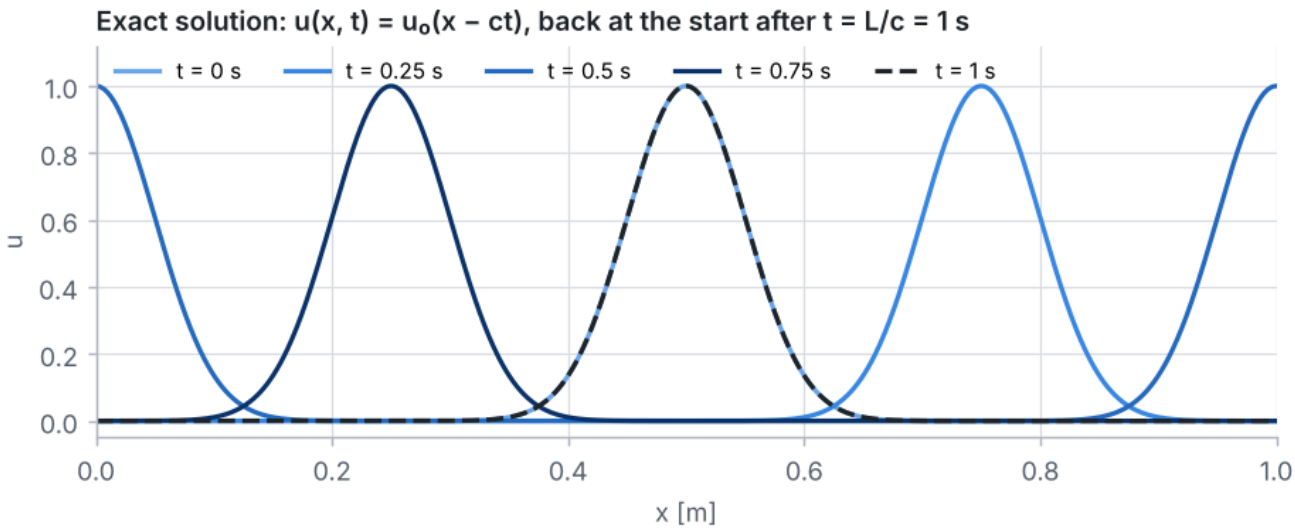
$$u(x,t) = u_0(x - ct).$$

After one lap the profile is exactly the initial one. Every change of shape we see is a **numerical** error. Information travels with speed c, in one direction only, and never fades: unlike the elliptic problems (1, 3–7), where every point feels every other point, and the parabolic Problem 2, where a disturbance is felt everywhere at once but dies out.

```
def u_exact(xx, t, pulse):
    """Exact solution: the initial pulse shifted by c t around the periodic loop."""
    return pulse((xx - c * t) % L)       # % L wraps x - c t back into [0, L)


for t in (0.0, 0.25, 1.0):
    print(f"t = {t:4.2f} s:  exact Gaussian at x = 0.50 and 0.75 m:  "
          f"{u_exact(0.50, t, gaussian):.4f}  {u_exact(0.75, t, gaussian):.4f}")
```

```
t = 0.00 s:  exact Gaussian at x = 0.50 and 0.75 m:  1.0000  0.0000
t = 0.25 s:  exact Gaussian at x = 0.50 and 0.75 m:  0.0000  1.0000
t = 1.00 s:  exact Gaussian at x = 0.50 and 0.75 m:  1.0000  0.0000
```



The exact Gaussian at five times: it moves round the loop (it crosses $x = L$ and re-enters at $x = 0$ at $t = 0.5$ s) and returns unchanged after $t = 1$ s.

Step 4 · Numerical scheme

All three schemes use a forward difference in time and the old values at $W = i - 1$, $P = i$, $E = i + 1$:

scheme	update formula	order
FTCS	$u_i^{n+1} = u_i^n - \frac{C}{2}\left(u_{i+1}^n - u_{i-1}^n\right)$	1 in t, 2 in x
upwind ($c > 0$)	$u_i^{n+1} = u_i^n - C\left(u_i^n - u_{i-1}^n\right) = (1 - C)\,u_i^n + C\,u_{i-1}^n$	1
Lax–Wendroff	$u_i^{n+1} = u_i^n - \frac{C}{2}\left(u_{i+1}^n - u_{i-1}^n\right) + \frac{C^2}{2}\left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right)$	2

Lax–Wendroff comes from the Taylor series $u^{n+1} = u^n + \Delta t\, u_t + \frac{1}{2}\Delta t^2\, u_{tt}$, with $u_t = -c\,u_x$ and $u_{tt} = c^2\,u_{xx}$ taken from the equation itself, and central differences for u_x and u_{xx}. In the code, Python's index `u[i - 1]` is `u[-1]` (the last node) for $i = 0$, and `u[(i + 1) % N]` is `u[0]` for $i = N - 1$: the periodic wrap costs nothing.

```python
def step_ftcs(u, C):
    """One time step of FTCS: forward in time, central in space."""
    N = len(u)
    new = [0.0] * N
    for i in range(N):
        W, P, E = u[i - 1], u[i], u[(i + 1) % N]     # periodic neighbours
        new[i] = P - C / 2 * (E - W)
    return new


def step_upwind(u, C):
    """One time step of first-order upwind (flow from left to right, c > 0)."""
    N = len(u)
    new = [0.0] * N
    for i in range(N):
        W, P = u[i - 1], u[i]                        # u[-1] is the last node: periodic
        new[i] = P - C * (P - W)
    return new

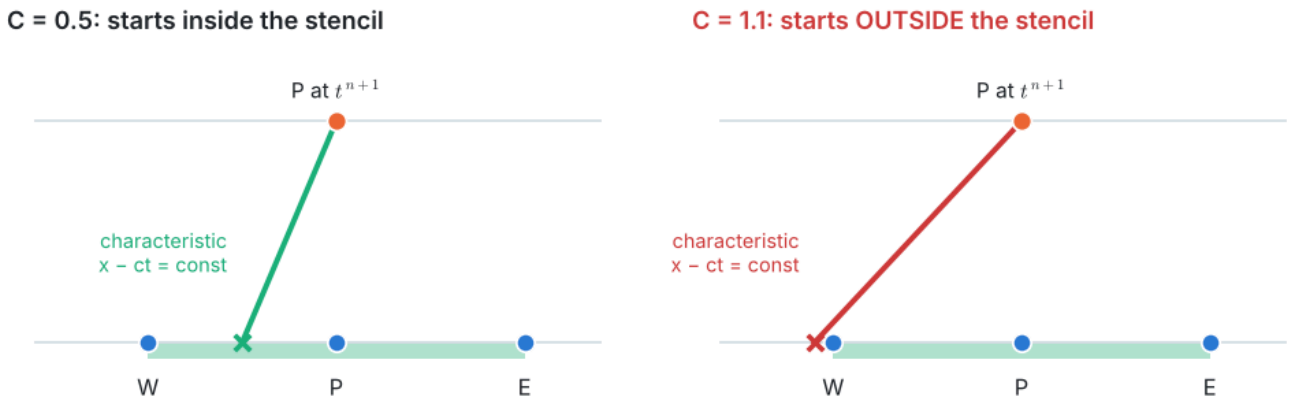

def step_lax_wendroff(u, C):
    """One time step of Lax-Wendroff: FTCS plus the term C^2/2 (E - 2P + W)."""
    N = len(u)
    new = [0.0] * N
    for i in range(N):
        W, P, E = u[i - 1], u[i], u[(i + 1) % N]
        new[i] = P - C / 2 * (E - W) + C**2 / 2 * (E - 2 * P + W)
    return new


schemes = {"FTCS": step_ftcs, "upwind": step_upwind, "Lax-Wendroff": step_lax_wendroff}
```

The CFL condition. The exact value at P at t^{n+1} comes along the characteristic through P, which started one step earlier at $x_P - C\,\Delta x$. If $C > 1$ it started beyond W, outside the stencil, and the scheme cannot see the information that decides the answer (Courant, Friedrichs and Lewy, 1928):

$$C = \frac{c\,\Delta t}{\Delta x} \le 1.$$

This is necessary, not sufficient.



The characteristic through P: at $C = 0.5$ it starts between W and P; at $C = 1.1$ it starts outside the stencil.

Von Neumann analysis. Follow one Fourier wave $u_i^n = G^n\, e^{\,i\theta i}$, with $\theta = k\,\Delta x$ the phase change per node ($\theta = \pi$: the shortest wave, $2\Delta x$ long). Putting it into a scheme and using $u_{i\pm1} = e^{\pm i\theta}\,u_i$ gives the **amplification factor** per time step:

$$|G_{FTCS}|^2 = 1 + C^2\sin^2\theta, \qquad |G_{upwind}|^2 = 1 - 2C(1-C)(1-\cos\theta), \qquad |G_{LW}|^2 = 1 - C^2(1-C^2)(1-\cos\theta)^2.$$

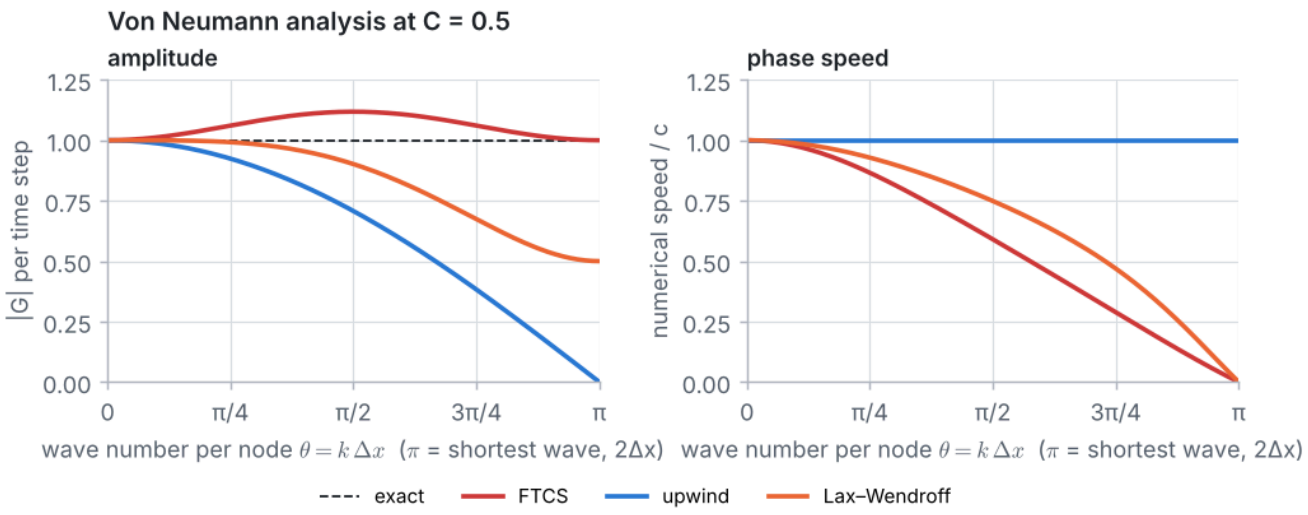
The exact solution only shifts every wave ($|G| = 1$). FTCS has $|G| > 1$ for every wave and **every** $C > 0$: it is unconditionally unstable. Upwind and Lax–Wendroff have $|G| \le 1$ exactly when $C \le 1$. The phase of G, divided by the exact shift $C\theta$, is the numerical speed of each wave.

```python
def amplification(scheme, C, theta):
    """Amplification factor G of one wave exp(i theta i) per time step (a complex number)."""
    if scheme == "FTCS":
        return 1 - 1j * C * math.sin(theta)
    if scheme == "upwind":
        return 1 - C * (1 - (math.cos(theta) - 1j * math.sin(theta)))      # e^{-i theta}
    return 1 - 1j * C * math.sin(theta) - C**2 * (1 - math.cos(theta))     # Lax-Wendroff


def speed_ratio(G, C, theta):
    """Numerical speed of the wave divided by c: -phase(G) / (C theta)."""
    return -math.atan2(G.imag, G.real) / (C * theta)


for C_test in (0.5, 1.0, 1.1):
    print(f"C = {C_test}:")
    for scheme in schemes:
        line = "   ".join(f"{abs(amplification(scheme, C_test, th)):.4f}"
                          for th in (math.pi / 4, math.pi / 2, math.pi))
        print(f"   {scheme:13s} |G| at theta = pi/4, pi/2, pi:   {line}")
```

```
C = 0.5:
   FTCS          |G| at theta = pi/4, pi/2, pi:   1.0607   1.1180   1.0000
   upwind        |G| at theta = pi/4, pi/2, pi:   0.9239   0.7071   0.0000
   Lax-Wendroff  |G| at theta = pi/4, pi/2, pi:   0.9919   0.9014   0.5000
C = 1.0:
   FTCS          |G| at theta = pi/4, pi/2, pi:   1.2247   1.4142   1.0000
   upwind        |G| at theta = pi/4, pi/2, pi:   1.0000   1.0000   1.0000
   Lax-Wendroff  |G| at theta = pi/4, pi/2, pi:   1.0000   1.0000   1.0000
C = 1.1:
   FTCS          |G| at theta = pi/4, pi/2, pi:   1.2669   1.4866   1.0000
   upwind        |G| at theta = pi/4, pi/2, pi:   1.0317   1.1045   1.2000
   Lax-Wendroff  |G| at theta = pi/4, pi/2, pi:   1.0108   1.1199   1.4200
```



At $C = 0.5$: FTCS amplifies every wave; upwind damps strongly (the $2\Delta x$ wave disappears in one step) but moves every wave at exactly c; Lax–Wendroff hardly damps, but its short waves are too slow.

Step 5 · Solution setup: explicit time marching

All three schemes are **explicit**: no system of equations, just one formula per node and step. Start from the initial pulse at the nodes and apply N/C steps for one lap. For $C = 1.1$ one lap is not a whole number of steps; we take the nearest number (91 steps, $t = 1.001$ s) and compare with the exact solution at that time.

```
def solve(pulse, scheme, N, C, t_end=t_lap, record=None):
    """March from u0 at the N nodes to t_end; returns the node positions, u and the final time.

    If record is a list, the largest |u| of every step is appended to it.
    """
    dx = L / N
    dt = C * dx / c
    n_steps = round(t_end / dt)
    step = schemes[scheme]
    x_nodes = [i * dx for i in range(N)]
    u = [pulse(xi) for xi in x_nodes]          # initial condition
    for n in range(n_steps):
        u = step(u, C)
        if record is not None:
            record.append(max(abs(v) for v in u))
    return x_nodes, u, n_steps * dt


def largest_error(x_nodes, u, t, pulse):
    """Largest difference from the exact solution at the nodes."""
    return max(abs(u[i] - u_exact(x_nodes[i], t, pulse)) for i in range(len(u)))
```

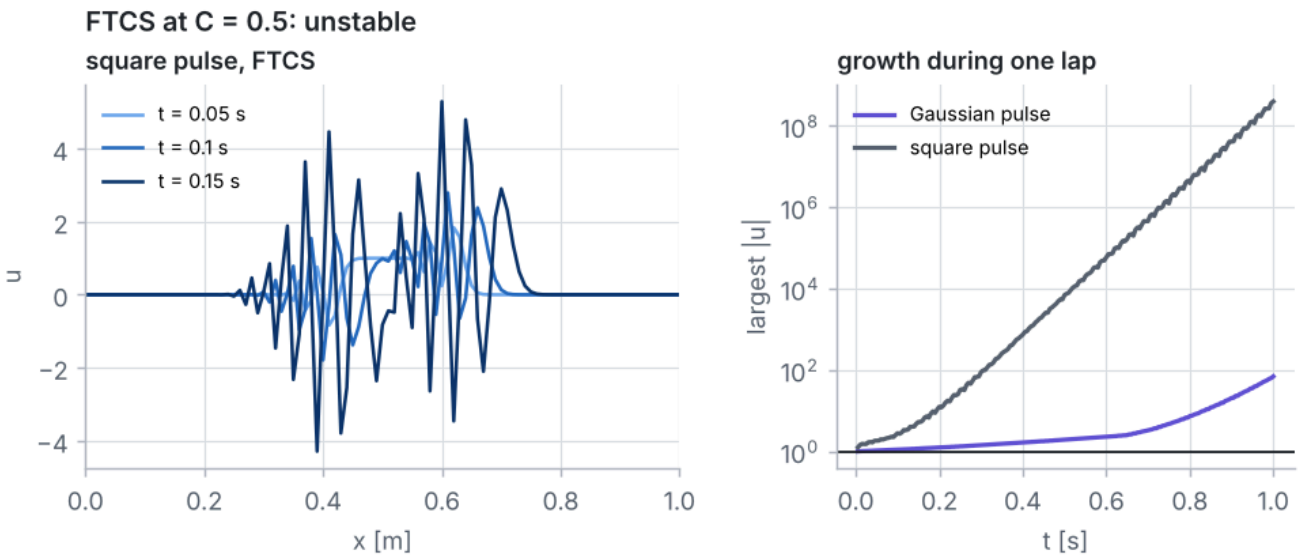
Step 6 · Compute

FTCS at $C = 0.5$:

```
ftcs_growth = {}
for name, pulse in pulses.items():
    record = []
    xs, u, t = solve(pulse, "FTCS", N, C, record=record)
    ftcs_growth[name] = ([(n + 1) * dt for n in range(len(record))], record)
    print(f"{name:15s} FTCS, C = {C}:  largest |u| after one lap = {record[-1]:.3g}"
          f"   (after 0.25 s: {record[n_lap // 4 - 1]:.3g})")

for C_small in (0.25, 0.1):
    xs, u, t = solve(gaussian, "FTCS", N, C_small)
    print(f"Gaussian pulse  FTCS, C = {C_small}: largest |u| after one lap = {max(abs(v) for v in u):.3g}")
```

```
Gaussian pulse  FTCS, C = 0.5:  largest |u| after one lap = 69.7   (after 0.25 s: 1.33)
square pulse    FTCS, C = 0.5:  largest |u| after one lap = 4.02e+08   (after 0.25 s: 32.5)
Gaussian pulse  FTCS, C = 0.25: largest |u| after one lap = 1.72
Gaussian pulse  FTCS, C = 0.1: largest |u| after one lap = 1.16
```



FTCS: wiggles grow at once on the square pulse; after one lap the largest value is 4×10^8. Even the smooth Gaussian grows to about 70.

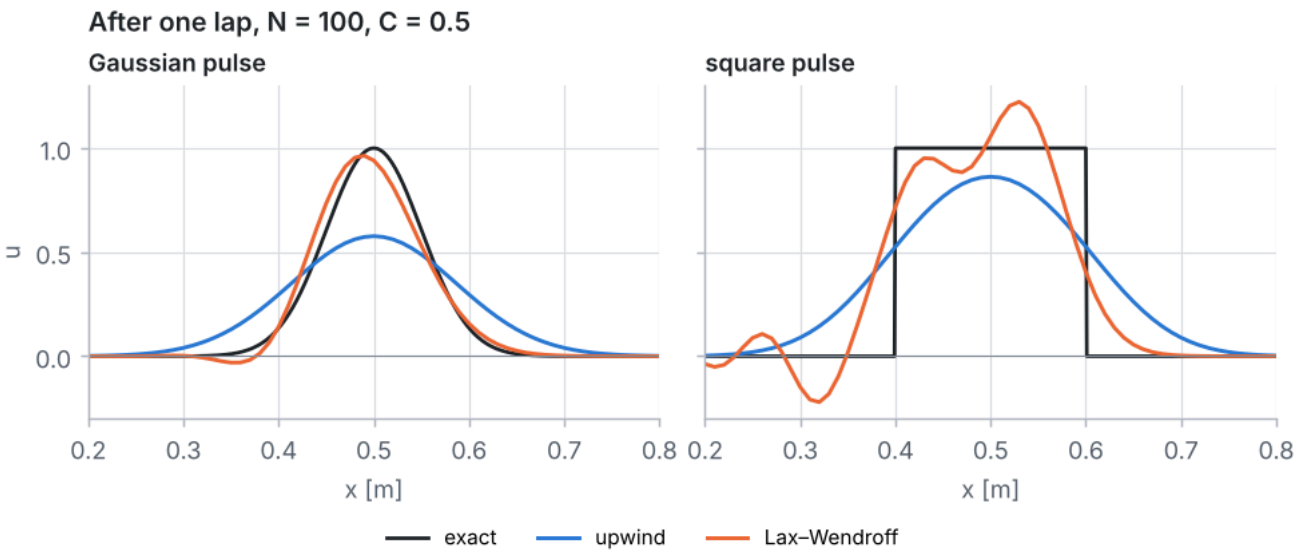
FTCS always blows up for advection. A smaller time step only slows the growth ($C = 0.1$ still reaches 1.16 after one lap, and grows without limit), because $|G| > 1$ for every $C > 0$. We drop FTCS from now on.

Upwind and Lax–Wendroff, one lap at $C = 0.5$:

```python
def report(scheme, N, C, pulse_names=pulses):
    """Run one lap and print peak, minimum and largest error for each pulse."""
    out = {}
    for name in pulse_names:
        xs, u, t = solve(pulses[name], scheme, N, C)
        err = largest_error(xs, u, t, pulses[name])
        out[name] = u
        print(f"   {scheme:12s}  {name:14s}  peak {max(u):9.4g}   min {min(u):9.4g}"
              f"   largest error {err:8.3g}   t = {t:.4g} s")
    return out


print(f"N = {N}, C = {C}, one lap ({n_lap} steps); exact: peak 1, min 0")
lap = {name: {} for name in pulses}
for scheme in ("upwind", "Lax-Wendroff"):
    result = report(scheme, N, C)
    for name in pulses:
        lap[name][scheme] = result[name]
```

```
N = 100, C = 0.5, one lap (200 steps); exact: peak 1, min 0
   upwind        Gaussian pulse  peak     0.577   min 5.612e-08   largest error    0.423   t = 1 s
   upwind        square pulse    peak    0.8626   min 1.507e-08   largest error    0.474   t = 1 s
   Lax-Wendroff  Gaussian pulse  peak    0.9639   min  -0.03069   largest error    0.129   t = 1 s
   Lax-Wendroff  square pulse    peak     1.223   min   -0.2207   largest error    0.596   t = 1 s
```



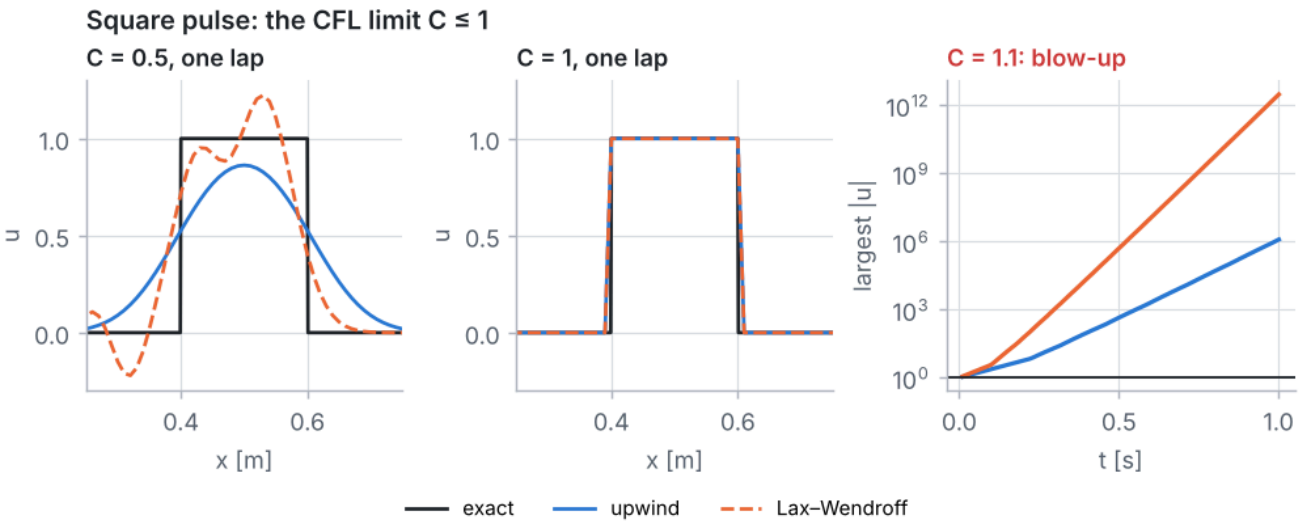
After one lap at $C = 0.5$: upwind keeps the pulses positive but smears them; Lax–Wendroff keeps the Gaussian, but wiggles trail behind the edges of the square pulse.

- **Upwind smears:** the Gaussian comes back at the right place but with **58 %** of its height; the square pulse becomes a bump with peak 0.86. No value is negative or above 1: the new value is an average $(1 - C)\,u_P + C\,u_W$ with positive weights (the boundedness argument of Problem 5). This is **numerical diffusion**.
- **Lax–Wendroff wiggles:** the Gaussian keeps **96 %** of its height with a small dip (-0.03) behind it; behind both edges of the square pulse a train of wiggles overshoots to 1.22 and undershoots to -0.22. The short waves travel too slowly and fall behind: **numerical dispersion**.

Courant numbers 1.0 and 1.1:

```
for C_run in (0.5, 1.0, 1.1):
    print(f"C = {C_run}:")
    for scheme in ("upwind", "Lax-Wendroff"):
        report(scheme, N, C_run)
```

```
C = 0.5:
   upwind        Gaussian pulse  peak     0.577   min 5.612e-08   largest error    0.423   t = 1 s
   upwind        square pulse    peak    0.8626   min 1.507e-08   largest error    0.474   t = 1 s
   Lax-Wendroff  Gaussian pulse  peak    0.9639   min  -0.03069   largest error    0.129   t = 1 s
   Lax-Wendroff  square pulse    peak     1.223   min   -0.2207   largest error    0.596   t = 1 s
C = 1.0:
   upwind        Gaussian pulse  peak         1   min 1.929e-22   largest error 6.66e-16   t = 1 s
   upwind        square pulse    peak         1   min         0   largest error        0   t = 1 s
   Lax-Wendroff  Gaussian pulse  peak         1   min 1.929e-22   largest error 6.66e-16   t = 1 s
   Lax-Wendroff  square pulse    peak         1   min         0   largest error        0   t = 1 s
C = 1.1:
   upwind        Gaussian pulse  peak     1.274   min -0.001333   largest error    0.279   t = 1.001 s
   upwind        square pulse    peak 1.227e+06   min -1.227e+06   largest error 1.23e+06   t = 1.001 s
   Lax-Wendroff  Gaussian pulse  peak     1.007   min -0.002363   largest error   0.0413   t = 1.001 s
   Lax-Wendroff  square pulse    peak 3.014e+12   min -3.011e+12   largest error 3.01e+12   t = 1.001 s
```



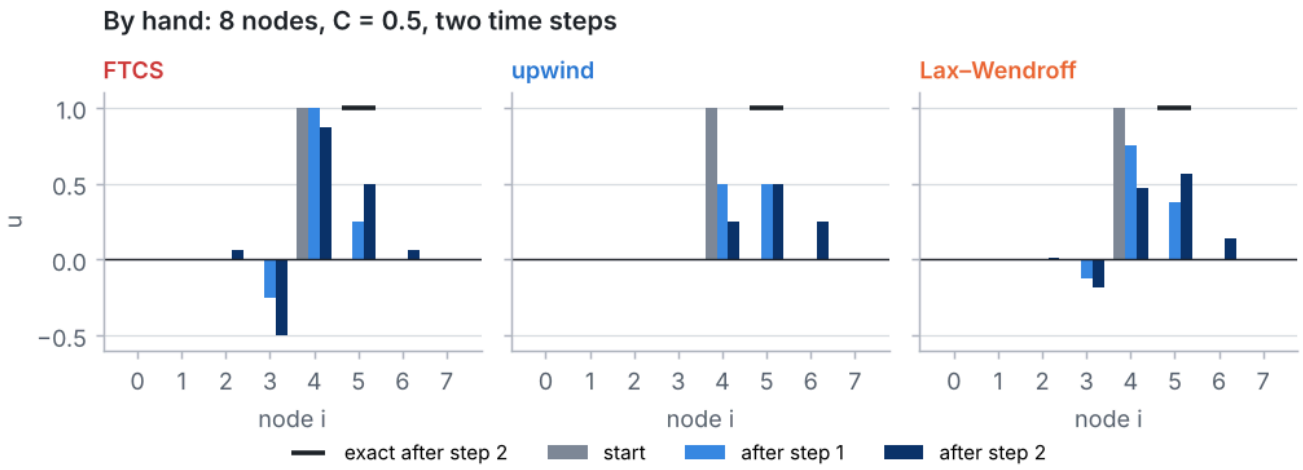
Square pulse: at $C = 1$ both schemes return it exactly; at $C = 1.1$ both blow up.

- $C = 1$: both formulas become $u_i^{n+1} = u_{i-1}^n$. Each step copies every value exactly one node downstream, which is what the flow does in $\Delta t = \Delta x / c$: the error is zero to round-off.
- $C = 1.1$: the CFL condition is violated. The shortest waves grow by $|G| = 1.2$ (upwind) and 1.42 (Lax–Wendroff) per step; the square pulse reaches 10^6 and 3×10^{12} within one lap. The Gaussian has almost no short waves, so after one lap its error is still 0.28 (upwind), but it grows without limit.

The whiteboard numbers. On $N = 8$ with $C = 0.5$ the square pulse is a single spike at node 4, and the three formulas have simple weights: FTCS $0.25\,u_W + u_P - 0.25\,u_E$, upwind $0.5\,u_W + 0.5\,u_P$, Lax–Wendroff $0.375\,u_W + 0.75\,u_P - 0.125\,u_E$. The exact answer after two steps is the spike at node 5.

```
N_hand, C_hand = 8, 0.5
x_hand = [i * L / N_hand for i in range(N_hand)]
u_start = [square(xi) for xi in x_hand]
hand_levels = {}
for scheme in schemes:
    levels = [u_start]
    for n in range(2):
        levels.append(schemes[scheme](levels[-1], C_hand))
    hand_levels[scheme] = levels
    print(f"{scheme}:")
    for n, u in enumerate(levels):
        values = "".join(f"{v:9.6g}" for v in u)
        print(f"   n = {n}: {values}   sum {sum(u):.6g}, sum of squares {sum(v * v for v in u):.6g}")
print("exact after 2 steps:", [square(xi - 2 * C_hand * L / N_hand) for xi in x_hand])
```

```
FTCS:
   n = 0:         0         0         0         0         1         0         0         0   sum 1, sum of squares 1
   n = 1:         0         0         0    -0.25         1      0.25         0         0   sum 1, sum of squares 1.125
   n = 2:         0         0    0.0625      -0.5     0.875       0.5    0.0625         0   sum 1, sum of squares 1.27344
upwind:
   n = 0:         0         0         0         0         1         0         0         0   sum 1, sum of squares 1
   n = 1:         0         0         0         0       0.5       0.5         0         0   sum 1, sum of squares 0.5
   n = 2:         0         0         0         0      0.25       0.5      0.25         0   sum 1, sum of squares 0.375
Lax-Wendroff:
   n = 0:         0         0         0         0         1         0         0         0   sum 1, sum of squares 1
   n = 1:         0         0         0    -0.125      0.75     0.375         0         0   sum 1, sum of squares 0.71875
   n = 2:         0         0 0.015625   -0.1875   0.46875    0.5625  0.140625         0   sum 1, sum of squares 0.591309
exact after 2 steps: [0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0]
```



Two steps by hand: upwind spreads the spike over three nodes, all positive; Lax–Wendroff puts a negative value behind it; FTCS makes the sum of squares grow.

All three keep $\sum u_i = 1$, but the **sum of squares** tells them apart: it grows for FTCS (1 → 1.125 → 1.273: unstable) and falls for upwind (1 → 0.5 → 0.375: strong damping) and Lax–Wendroff (1 → 0.719 → 0.591).

Step 7 · Post-processing and verification

Upwind: the modified equation. Taylor-expanding u_i^{n+1} and u_{i-1}^n about (x_i, t^n) and using $u_{tt} = c^2 u_{xx}$ shows that the upwind scheme actually solves

$$u_t + c\,u_x = D_{num}\,u_{xx}, \qquad D_{num} = \frac{c\,\Delta x}{2}\,(1 - C),$$

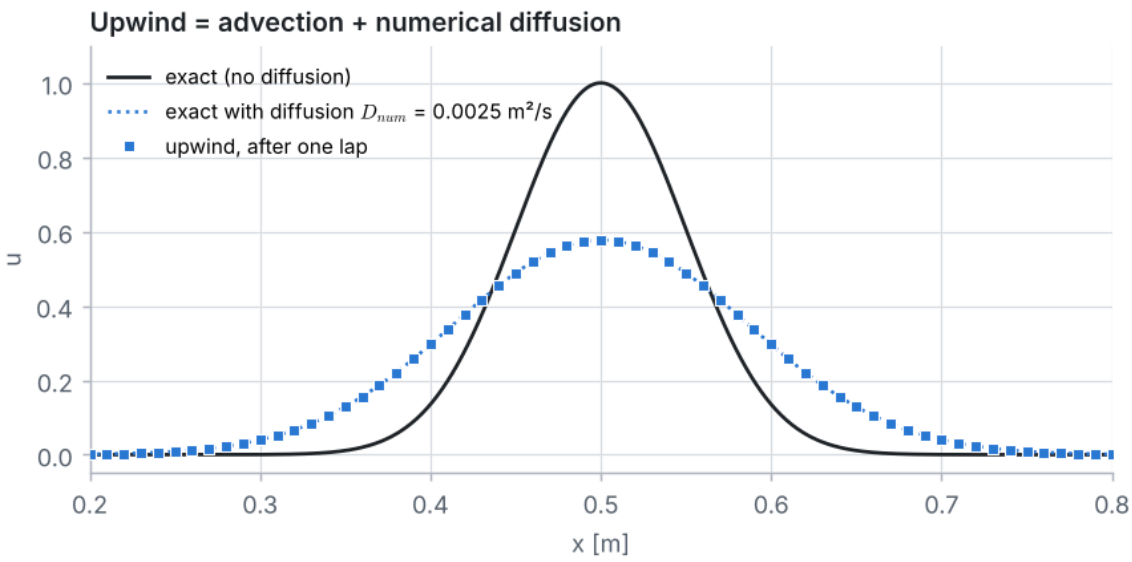
up to higher-order terms. This is the false diffusion of Problem 5, $u\,\Delta x/2$, times $(1 - C)$: in an unsteady calculation the error of the forward time step cancels part of it, all of it at $C = 1$. (For FTCS the same expansion gives $D_{num} = -\frac{1}{2}c^2\,\Delta t < 0$: a negative diffusion, another way to see its instability.)

A Gaussian that diffuses with D for a time t keeps its area and widens to $\sigma_t^2 = \sigma^2 + 2Dt$, so its peak falls to σ/σ_t. This **predicts** the upwind peak without running the scheme:

```python
def upwind_peak_predicted(N, C, t=t_lap):
    """Peak of the Gaussian after time t predicted by the modified equation of upwind."""
    D_num = c * (L / N) / 2 * (1 - C)
    return sigma / math.sqrt(sigma**2 + 2 * D_num * t), D_num


for C_run in (0.25, 0.5, 0.75, 1.0):
    predicted, D_num = upwind_peak_predicted(N, C_run)
    xs, u, t = solve(gaussian, "upwind", N, C_run)
    print(f"C = {C_run:4.2f}:  D_num = {D_num:.6f} m^2/s   predicted peak = {predicted:.4f}"
          f"   upwind peak = {max(u):.4f}")
```

```
C = 0.25:  D_num = 0.003750 m^2/s   predicted peak = 0.5000   upwind peak = 0.4998
C = 0.50:  D_num = 0.002500 m^2/s   predicted peak = 0.5774   upwind peak = 0.5770
C = 0.75:  D_num = 0.001250 m^2/s   predicted peak = 0.7071   upwind peak = 0.7073
C = 1.00:  D_num = 0.000000 m^2/s   predicted peak = 1.0000   upwind peak = 1.0000
```



The upwind points after one lap lie on the exact solution of advection *with* diffusion $D_{num} = 0.0025$ m²/s.

The prediction matches the computed peak to within 0.0005. $D_{num} = 0.0025$ m²/s is 2.5 million times the real diffusion of the dye. And on a fixed grid a **smaller** time step makes upwind **worse** ($C = 0.25$: peak 0.50).

Lax–Wendroff instead solves $u_t + c\,u_x = -\frac{1}{6}c\,\Delta x^2\,(1 - C^2)\,u_{xxx} + \ldots$: the leading error is a third derivative, which changes the speed of the waves (dispersion) instead of damping them.

Conservation. All three schemes have the form "new = old − (flux out − flux in)", so the amount of dye $\sum u_i\,\Delta x$ is kept exactly:

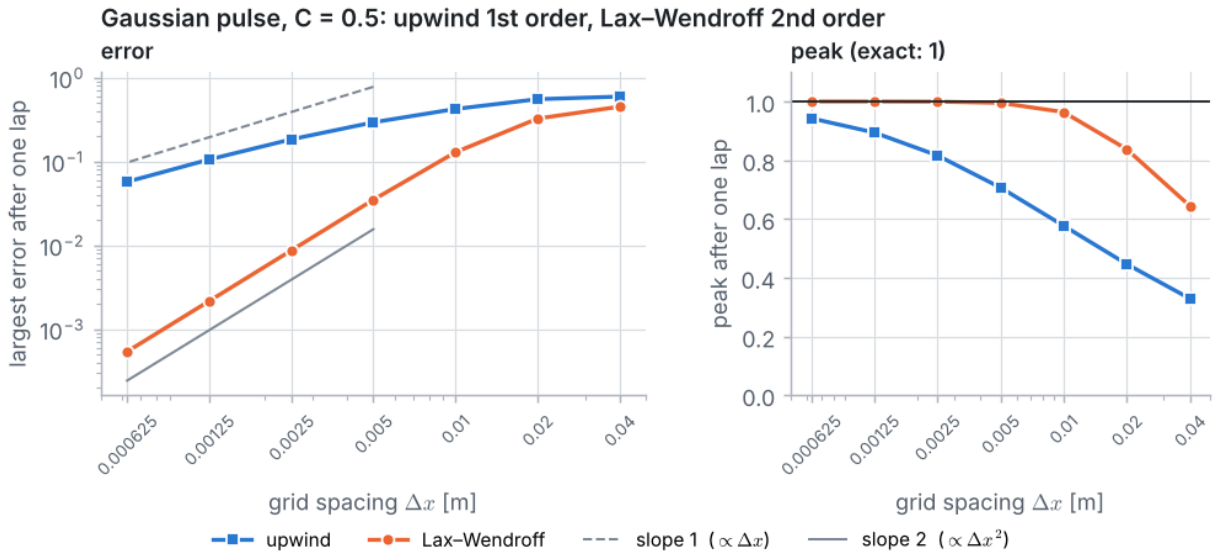
```
for name, pulse in pulses.items():
    total_start = sum(pulse(xi) for xi in x) * dx
    line = "   ".join(f"{scheme}: {sum(lap[name][scheme]) * dx:.6f}" for scheme in lap[name])
    print(f"{name:15s} amount of dye at t = 0: {total_start:.6f}   after one lap:  {line}")
```

```
Gaussian pulse  amount of dye at t = 0: 0.125331   after one lap:  upwind: 0.125331   Lax-Wendroff: 0.125331
square pulse    amount of dye at t = 0: 0.210000   after one lap:  upwind: 0.210000   Lax-Wendroff: 0.210000
```

Error study. Gaussian pulse, $C = 0.5$, one lap, $N = 25$ to 1600; the observed order between two grids is $p = \log(e_{coarse}/e_{fine})/\log 2$:

```
grids = [25, 50, 100, 200, 400, 800, 1600]
dxs, errors, peaks = [], {"upwind": [], "Lax-Wendroff": []}, {"upwind": [], "Lax-Wendroff": []}
start = time.time()
print("                |              upwind               |      Lax-Wendroff")
print("     N   steps  |    error  order    peak (theory)  |    error  order     peak")
for n_int in grids:
    line = f"{n_int:6d}  {round(n_int / C):6d}  |"
    for scheme in ("upwind", "Lax-Wendroff"):
        xs, u, t = solve(gaussian, scheme, n_int, C)
        error = largest_error(xs, u, t, gaussian)
        e = errors[scheme]
        order = f"{math.log(e[-1] / error) / math.log(2):5.2f}" if e else "     "
        e.append(error)
        peaks[scheme].append(max(u))
        if scheme == "upwind":
            line += f" {error:8.2e}  {order}  {max(u):.4f} ({upwind_peak_predicted(n_int, C)[0]:.4f})  |"
        else:
            line += f" {error:8.2e}  {order}  {max(u):.5f}"
    dxs.append(L / n_int)
    print(line)
print(f"run time: {time.time() - start:.1f} s")
```

```
              |              upwind               |      Lax-Wendroff
     N   steps  |    error  order    peak (theory)  |    error  order     peak
    25      50  | 5.94e-01         0.3291 (0.3333)  | 4.50e-01         0.64436
    50     100  | 5.54e-01   0.10  0.4465 (0.4472)  | 3.25e-01   0.47  0.83755
   100     200  | 4.23e-01   0.39  0.5770 (0.5774)  | 1.29e-01   1.33  0.96386
   200     400  | 2.93e-01   0.53  0.7070 (0.7071)  | 3.49e-02   1.88  0.99521
   400     800  | 1.84e-01   0.67  0.8165 (0.8165)  | 8.68e-03   2.01  0.99936
   800    1600  | 1.06e-01   0.80  0.8944 (0.8944)  | 2.16e-03   2.01  0.99994
  1600    3200  | 5.72e-02   0.88  0.9428 (0.9428)  | 5.39e-04   2.00  0.99999
run time: 2.7 s
```



Largest error and peak after one lap. Lax–Wendroff falls along slope 2; upwind approaches slope 1 slowly.

- **Lax–Wendroff** converges at **2nd order** (observed 2.0 from $N = 200$ on) and keeps 99.9 % of the peak on 400 intervals.
- **Upwind** converges at only **1st order**, and reaches it slowly (0.88 between $N = 800$ and 1600), because its error is the peak loss of the modified equation, which is proportional to Δx only once $D_{num}\, t \ll \sigma^2$. On 1600 intervals it keeps 94 % of the peak: less than Lax–Wendroff on 100 intervals, with 256 times less work.

What this means for CFD codes. Every unsteady flow calculation has an advection term and faces the same choice: first-order upwinding is bounded but smears fronts and vortices; second-order schemes are accurate but wiggle at sharp changes. Practical codes use high-resolution (TVD, flux-limited) schemes or controlled stabilisation, and the time integrator matters too: the implicit BDF methods used by default in COMSOL are stable for any time step, but they damp waves, like upwinding.

Result ($N = 100$, one lap). FTCS blows up at every Courant number. At $C = 0.5$ upwind returns the Gaussian with peak **0.577** (predicted by the modified equation: 0.577) and the square pulse with peak 0.86, smooth and positive; Lax–Wendroff returns the Gaussian with peak **0.964** and the square pulse with wiggles between -0.22 and 1.22 behind its edges. At $C = 1$ both are exact; at $C = 1.1$ both blow up. On refined grids Lax–Wendroff converges at **2nd order**, upwind at **1st order**.

Key ideas

- The advection equation is **hyperbolic**: u is carried unchanged along the characteristics $x - ct = \text{const}$.
- The **Courant number** $C = c\,\Delta t/\Delta x$ is the distance the flow moves in one step, in grid intervals; explicit three-point schemes need $C \le 1$ (**CFL condition**).
- **Von Neumann analysis**: FTCS is unconditionally unstable; upwind and Lax–Wendroff are stable for $C \le 1$, exact at $C = 1$.
- Upwind adds **numerical diffusion** $D_{num} = \frac{1}{2} c\,\Delta x\,(1 - C)$ (1st order, bounded, smeared); Lax–Wendroff is **dispersive** (2nd order, wiggles behind sharp fronts).
- A scheme can conserve the dye exactly and still be badly wrong.